

Data Structure Lab10 : Trees and Binary Trees

2024-2025

Topics

1. Create Position Interface
2. Create Tree Interface
3. Create AbstractTree Class.
4. Create BinaryTree Interface
5. Create AbstractBinaryTree Class
6. Create Linked Binary Tree Using Linked List structure (Nodes)
7. Implement Basic Methods of LinkedBinaryTree
 - addRoot(E e)
 - addLeft(Position<E> p ,E e)
 - addRight(Position<E> p ,E e)
 - set(Position<E> p ,E e)
 - remove(Position<E> p)

Homework

1. Implement the Array Based Binary Tree Data structure as it is described in chapter 8.

```
public class ArrayBinaryTree<E> {
    private E[] tree;
    private int size;
    private static final int DEFAULT_CAPACITY = 10;

    public ArrayBinaryTree() {
        tree = (E[]) new Object[DEFAULT_CAPACITY];
        size = 0;
    }

    public void add(E element) {
```

Data Structure Lab10 : Trees and Binary Trees

2024-2025

```
        ensureCapacity();
        tree[size++] = element; // Add the element in the next available
position
    }
```

```
    public E get(int index) {
        if (index < 0 || index >= size) throw new
IndexOutOfBoundsException();
        return tree[index];
    }
```

```
    public int leftChildIndex(int index) {
        return 2 * index + 1; // Calculate left child index
    }
```

```
    public int rightChildIndex(int index) {
        return 2 * index + 2; // Calculate right child index
    }
```

```
    public int parentIndex(int index) {
        if (index == 0) throw new IllegalArgumentException("Root node
has no parent.");
        return (index - 1) / 2; // Calculate parent index
    }
```

```
    private void ensureCapacity() {
        if (size == tree.length) {
            tree = Arrays.copyOf(tree, tree.length * 2);
        }
    }
```

```
    public int size() {
```

Data Structure Lab10 : Trees and Binary Trees

2024-2025

```
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }
}
```